



on Information and Systems

DOI:10.1587/transinf.2025KBP0015

Publicized:2026/02/13

This article has been accepted and published on J-STAGE in advance of copyediting. Content is final as presented.



A PUBLICATION OF THE INFORMATION AND SYSTEMS SOCIETY

The Institute of Electronics, Information and Communication Engineers

Kikai-Shinko-Kaikan Bldg., 5-8, Shibakoen 3 chome, Minato-ku, TOKYO, 105-0011 JAPAN

Evaluating the Resilience of Obfuscated Code Against LLM-based Function Name Recovery in MATE Attacks

Tetsuya KITAOKA^{†a)}, Nonmember, Yuichiro KANZAKI^{††b)}, Takashi ISHIO^{†††c)}, Members,
Kazumasa SHIMARI^{†d)}, Nonmember, and Kenichi MATSUMOTO^{†e)}, Fellow

SUMMARY In recent years, improvements in **decompilers and large language models (LLMs)** have made it possible to use powerful techniques for analyzing **software binaries**, including **automated function name recovery**. Such powerful binary analysis techniques can be exploited for **Man-At-The-End (MATE) attacks**, posing a serious threat to the software ecosystem. To protect software against MATE attacks, **software obfuscation is widely used**. However, it is **not clear** how effectively **code obfuscation can prevent LLM-based MATE attacks**, and thus a method for evaluating the resilience of obfuscated code against such attacks is required. This paper proposes a framework for evaluating the resilience of obfuscated code against LLM-based function name recovery MATE attacks. To construct the framework, **resilience against the LLM-based function name recovery attack** is characterized. The resilience is defined based on the **morphological or semantic similarity** between the original function name and the recovered function names. In the experiment, the **resilience of five well-known obfuscation methods against LLM-based function name recovery attacks** (i.e., how effectively these obfuscation methods prevent the prediction of appropriate function names) using the Gemma3-27B-IT, gpt-oss-20b, and llama-4-scout-17b-16e-instruct LLM general-purpose models is evaluated. The results reveal that **the obfuscation method based on code virtualization has the highest resilience**, while the method based on **encoding arithmetic has the lowest resilience**. In addition, the resilience of some obfuscation methods varies depending on the model. Furthermore, the analysis of high-resilience obfuscated code indicates that **LLM-recovered function names for functions evaluated as high resilience are relatively less related** to the implementation of the original functions. This paper makes two contributions: **the development of a framework for evaluating the resilience of obfuscated code against LLM-based function name recovery MATE attacks**, and an **experimental evaluation of the resilience of existing obfuscation methods against such attacks**.

key words: Software Protection, Code Obfuscation, Large Language Model, Man-At-The-End Attack

1. Introduction

In recent years, improvements in the performance of decompilers and the emergence of large language models (LLMs) have **significantly accelerated the development of techniques for analyzing binary executables**. A decompiler is a tool that converts a **binary executable into readable pseudocode** (e.g., a C-like pseudocode text [1]). Many binary analysis

tools, such as IDA [1], Ghidra [2], and Binary Ninja [3], **possess their own decompiler implementations**, and are widely available to end users. In addition, LLMs, such as **OpenAI's GPT family**, are **rapidly becoming popular** and are considered useful for assisting human program comprehension [4].

Combining recent decompilers with LLMs, a new method of analyzing binaries is emerging [5, 6]. In this method, **the binary is first decompiled into pseudocode** by a decompiler, **and then the code is analyzed by an LLM to recover function names and/or generate a summary** of the original binary. While this analysis method is useful for malware analysis, **it can also be exploited for Man-At-The-End (MATE) attacks**, i.e., attempts to obtain assets from binaries **or tamper with code by malicious end users** who have control over the device on which the binaries are executed [7]. This poses **a serious concern for the software ecosystem**, and therefore **effective defenses against such attacks are required**.

One major technique to counter MATE attacks is **software obfuscation**. Software obfuscation transforms the representation and structure of program code to **make it difficult to analyze**, while preserving its functional equivalence [8, 9]. While **the resilience of software obfuscation to human attacks [10] and automated attacks based on symbolic execution [11, 12] has been widely discussed**, it is not clear how resilient existing obfuscation methods are **against the LLM-based MATE attacks**.

This paper aims to propose a framework for evaluating the resilience of obfuscated code against LLM-based function name recovery MATE attacks. A function name recovery attack is the task **of predicting function names based on the logic of functions** in a binary executable, from which **symbol information has been stripped**. If the original function name is recovered by this attack, the attacker **may obtain significant clues for identifying the parts of the program** that should to be analyzed or modified, as well as for **understanding the code itself**. Thus, preventing the recovery of function names is important for protecting against MATE attacks.

This paper addresses the following research question (RQ): **how resilient are software-based obfuscation methods against LLM-based function name recovery in MATE attacks?** In other words, this paper aims to address the question **of what degree common obfuscation methods can prevent LLM-based function name recovery attacks**. To answer this question, we first **characterize the LLM-based function name recovery attack and the resilience** against it, and then **quantitatively measure the resilience** of well-known obfusca-

[†]Nara Institute of Science and Technology, Nara, 630-0192, Japan

^{††}National Institute of Technology, Kumamoto College, Kumamoto, 861-1102, Japan

^{†††}Future University Hakodate, Hokkaido, 041-8655 Japan

a) E-mail: kitaoka.tetsuya.ks9@is.naist.jp

b) E-mail: kanzaki@kumamoto-nct.ac.jp

c) E-mail: ishio@fun.ac.jp

d) E-mail: k.shimari@is.naist.jp

e) E-mail: matumoto@is.naist.jp

tion methods. The experimental data reported in this paper are available on [Zenodo](https://doi.org/10.5281/zenodo.17025135).[†]

The remainder of this paper is organized as follows: Sect. 2 reviews related work. Section 3 explains the proposed evaluation framework. Section 4 describes an experiment to evaluate the resilience of existing obfuscation methods against LLM-based function name recovery attacks. Section 5 discusses the effectiveness of the proposed method and the practical applicability of the proposed framework. Section 6 discusses the threats to the validity of our findings. Finally, Sect. 7 summarizes this paper and discusses future work.

2. Related Work

2.1 Software obfuscation methods and quality evaluation

Software obfuscation has been widely used for protecting security-sensitive code and data within software against MATE attacks. Well-known obfuscation methods include encoding literals, encoding arithmetic, inserting opaque predicates, flattening control flows, and code virtualization. These obfuscation methods are presented in the literature [8, 9]. Furthermore, tools implementing these methods, such as Tigress [13], exist in the open community.

In recent years, research evaluating the quality of obfuscation methods has been actively conducted from various approaches. The quality of obfuscation methods can be evaluated based on well-known quality aspects, namely potency, resilience, stealth, and cost, proposed by Collberg et al. [8]. Specific metrics for evaluating these quality aspects have also been proposed. Examples include evaluating the potency of obfuscated code based on the time required for human analysis [10], evaluating the resilience of obfuscated code against symbolic execution attacks [11], and evaluating the stealth of obfuscated code using an N-gram model [14]. Ebad et al. [15] summarize commonly used metrics for each quality aspect.

De Sutter et al. [7] comprehensively survey how the quality of obfuscation methods has been evaluated in the literature. They redefined the resilience of an obfuscation method as its capability to resist targeted attacks to undo it or otherwise nullify or reduce its impact [7]. Our study focuses on the resilience of code obfuscation against automated MATE attacks.

2.2 Evaluating resilience to automated MATE attacks

The resilience of obfuscation methods can be evaluated based on whether MATE attacks succeed. MATE attackers trying to extract internal assets from software distributed as binaries can use various binary analysis tools, including the latest disassemblers and decompilers.

Understanding disassembled or decompiled code obtained from obfuscated binaries requires significant effort

for MATE attackers. To minimize this effort, automated MATE attacks depending on their goals have been developed. Examples of existing automated MATE attacks include those using symbolic execution or machine learning. Therefore, obfuscation methods are expected to be resilient against such automated MATE attacks. Banescu et al. [11] addressed the problem of evaluating the resilience of code obfuscation transformations against automated MATE attacks that use symbolic execution. Their study characterized resilience based on the time required for symbolic execution tools to generate test cases or find inputs that reach specific paths within the program. In addition, Banescu et al. [12] proposed an approach that uses machine learning for predicting the resilience of obfuscating transformations against automated attacks. Furthermore, Isobe et al. [16] evaluated the resilience of Java bytecode to which the identifier renaming method (i.e., name obfuscation) had been applied against automated attacks that aim to recover method names using a random forest model.

Although LLM-based automated MATE attacks are now easily available for general end users, the resilience of obfuscation methods against such attacks has not been widely discussed. To complement existing research, this study aims to examine the resilience of well-known obfuscation methods against LLM-based attacks.

2.3 Analyzing code using LLMs

In recent years, several studies have been conducted to evaluate the ability of LLMs to support humans in analyzing binary code. Shang et al. investigated and compared the ability of LLMs to understand binary code [5]. Specifically, they evaluated how well existing LLMs can understand pseudocode obtained by decompiling a binary, focusing on two aspects: function name recovery and binary code summarization. This research served as one of the inspirations for our study. Providing decompiled code to an LLM and trying to retrieve information useful for understanding the binary is also addressed in our study. While Shang et al.'s study focuses on non-obfuscated binaries [5], our study targets obfuscated binaries and quantitatively measures resilience against LLM-based function name recovery attacks, based on the similarity between the original and the recovered function names.

Jiang et al. proposed an LLM that infers function names from decompiled code obtained from binaries [6]. In part of their experiments, they evaluated the model on obfuscated binaries using three methods: bogus control flow, control flow flattening, and instruction substitution. While Jiang et al. focused on developing a model specialized for function name inference, our approach investigates how five well-known obfuscation methods are able to make it hard to infer function names when using an easily accessible general-purpose LLM.

Patsakis et al. evaluated the capability of LLMs by executing the task of identifying URLs that download malware (Emotet) from obfuscated Powershell payloads [17]. Our

[†]<https://doi.org/10.5281/zenodo.17025135>

proposed method targets pseudocode obtained by decompiling obfuscated binaries, and evaluates the capabilities of LLMs by executing the task of inferring original function names from function implementations.

Beste et al. conducted experiments on de-obfuscation through an LLM task that transforms given obfuscated code into a simpler and semantically equivalent version [18]. Our proposed method focuses on evaluating the capability of an LLM to recover function names that support a MATE attacker’s understanding, without simplifying the code.

Fang et al. evaluated how well LLMs can understand obfuscated JavaScript code [19]. In their experiments, obfuscation methods such as string replacement, string splitting, dead code injection, and control flow flattening were used. While Fang et al. target JavaScript source code, our proposed method evaluates the resilience in a scenario that involves understanding pseudocode text decompiled from obfuscated binaries.

2.4 Predicting function names from code

Task of predicting function or method names from their implementations has been discussed in prior research.

Kashiwabara et al. proposed a verb recommendation method for Java method name from Java method bodies [20]. They used association rules focusing on the relationships between verbs in method names and identifiers within methods. Our proposed method evaluates how accurately the original function names can be predicted using a combination of a decompiler and an LLM when applied to stripped binary that contain no identifier information.

Gao et al. proposed a framework for reassigning function names targeting stripped binaries [21]. Their framework predicts function names based on assembly instruction sequences and control flow information obtained by disassembling the code. Our proposed method predicts function names using an LLM from pseudocode text obtained by decompiling binaries.

When these methods are fully developed and become widely adopted, they may serve as an evaluation criterion for obfuscation methods.

3. Evaluation Framework

3.1 Overview

Figure 1 shows an overview of the proposed evaluation framework. The proposed framework includes an obfuscated binary generation process and function name recovery attacks using a decompiler and an LLM. In this framework, resilience of obfuscated code is calculated based on the similarity between the original function names, which are known only to developers, and the function names recovered by the attack.

To formalize this evaluation framework, we define the following programs and function names:

- p represents the original program in a dataset.

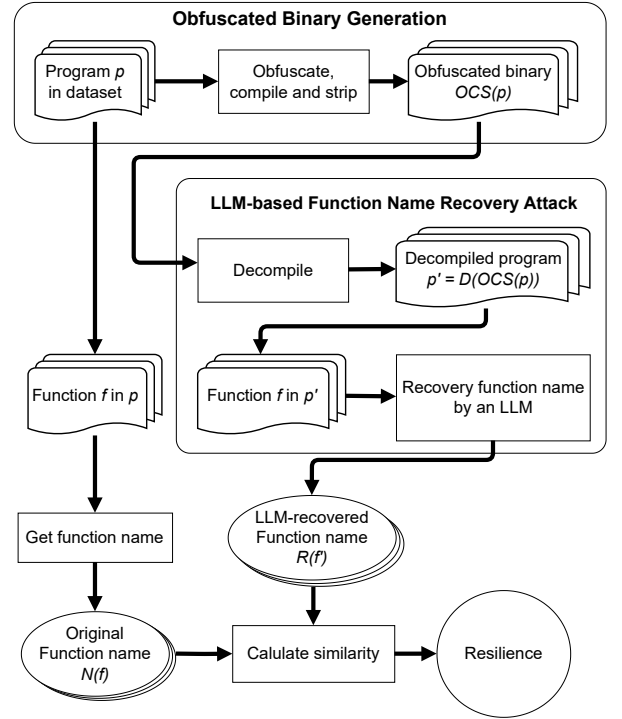


Fig. 1: Overview of the proposed evaluation framework

- $OCS(p)$ represents a binary program obtained by applying obfuscating transformations, compiling the code, and stripping symbol tables.
- $D(b)$ represents a program (a pseudocode text) obtained by decompiling a binary b .
- $R(f)$ represents a function name recovered by an LLM from the decompiled code of a function f .
- $N(f)$ is the function name of f .

Our evaluation framework takes as input a set of programs for benchmarking. First, it translates each program p into its obfuscated version $p' = D(OCS(p))$ by applying obfuscating transformations and decompilation. While the resultant pseudocode p' still has the same functionalities as the original program p , functions in p' have no meaningful function names. Next, the framework applies an LLM to recover the function names from their decompiled code in p' . While this is not generally guaranteed, some obfuscating transformations preserves functional units in the programs. Hence, the framework can obtain a pair of a function f in p and its obfuscated version f' in p' . If the obfuscating transformations are resilient to the function name recovery, the recovered function name $R(f')$ should be different from the name $N(f)$. Hence, the framework measures the similarity between $R(f')$ and $N(f)$ for each pair of functions in p' and p to evaluate the resilience of the obfuscating transformations.

To quantitatively assess resilience to function name recovery attacks, we employ a similarity metric. Given a func-

tion f in a program p , we identify its obfuscated version f' in $p' = D(OCS(p))$. Resilience for a function f is defined as:

$$Resilience(f) = 1 - Similarity(R(f'), N(f))$$

where *Similarity* represents the degree of similarity between two function names. The meaning of *Resilience* depends on the similarity metric employed.

3.2 Metric

In this paper, we propose a similarity metric that takes into account both morphology and semantics of function names. This is because function names generated by an LLM are not guaranteed to match the original names at the word level. For example, a function originally named `binary_search` may be inferred as a more general name such as `find_string`, depending on the model's interpretation of the code. Even though the inferred function names do not share any words with the originals, they can still provide useful clues for understanding the intent of the code.

In the following subsections, we describe morphological similarity and semantic similarity between sets of words, and then define our similarity metric based on these two similarities. To formalize these measures, we use $T(n)$ as a set of words obtained from the function name n by recognizing their styles such as snake case and camel case. For example, when n is `binary_search`, $T(n)$ is {"binary", "search"}, and when n is `getBucketIndex`, $T(n)$ is {"get", "bucket", "index"}.

3.2.1 Morphological Similarity

Morphological similarity focuses on the shape of each word. Generally, words used in function names are often derives or abbreviations that have the same base word. When at least one morphologically similar token in two function names, such tokens may provide significant clues for code analysis. For example, when the LLM-recovered function name is `convert_to_quaternion` and its original name is `quat_from_euler`, "quaternion" and "quat" are morphologically similar tokens. Thus, these function names are determined to be morphologically similar.

In order to determine whether two tokens are morphologically similar, we adapt part of the method proposed by Gao et al. [21], which aims to find semantically similar tokens. Specifically, whether two tokens, t_1 and t_2 , are morphologically similar is determined by the following definition of *IsMorphologicallySimilar*(t_1, t_2):

$$IsMorphologicallySimilar(t_1, t_2) = \begin{cases} \text{True} & \text{if } t_1 \text{ starts with } t_2 \\ & \text{or } t_2 \text{ starts with } t_1 \\ LS(t_1, t_2) > 0.6 & \text{if } t_1[0] \text{ equals } t_2[0] \\ & \text{or } t_1[-1] \text{ equals } t_2[-1] \\ \text{False} & \text{otherwise} \end{cases}$$

where $t[0]$ and $t[-1]$ indicate the first and the last character of a token t , respectively, and $LS(t_1, t_2)$ represents the Levenshtein similarity [22] between t_1 and t_2 .

The morphological similarity between two function names, n_1 and n_2 , is defined as follows:

$$MorphoSim(n_1, n_2) = \begin{cases} 1 & \text{if } \exists t_1 \in T(n_1), \exists t_2 \in T(n_2) \\ & \text{s.t. } IsMorphologicallySimilar(t_1, t_2) \\ 0 & \text{otherwise} \end{cases}$$

If any tokens are morphologically similar between n_1 and n_2 , they are considered morphologically similar functions.

3.2.2 Semantic Similarity

Semantic similarity focuses on the meanings of words. Function names recovered by LLM may contain words that are semantically similar to words used in the original function names. For example, the verbs "collect" and "gather" are semantically similar and the nouns "client" and "guest" are also semantically similar.

The semantic similarity between two tokens t_1 and t_2 is measured using path similarity [23]. Path similarity is a metric that measures the relatedness of two word senses by finding shortest path in an *is-a* hierarchy [23, 24]. The semantic similarity for tokens $SemSim_t$ is defined using the maximum path similarity of all pairs of synsets obtained from tokens, as follows:

$$SemSim_t(t_1, t_2) = \max \left\{ \frac{1}{lsp(s_1, s_2) + 1} \mid \begin{array}{l} s_1 \in synsets(t_1), \\ s_2 \in synsets(t_2) \end{array} \right\}$$

where $synsets(t)$ represents the set of synsets obtained from the token t , and $lsp(s_1, s_2)$ is the length of the shortest path between s_1 and s_2 , respectively. A synset is a set of synonyms sharing a common meaning [25] used in WordNet [26].

In our metric, we focus on verbs and nouns contained in function names. The reason we focus on verbs is that we referred to Isobe et al.'s work [16], which calculates the similarity between verbs contained in the method names of Java programs using WordNet. The reason we also focus on nouns is that function names representing algorithms, such as "quick_sort" or "binarySearch", often contain not only verbs but also characteristic nouns. The semantic similarity between verbs and the semantic similarity between nouns in two function names, n_1 and n_2 are defined as follows.

$$VerbSim(n_1, n_2) = \max \left\{ SemSim_t(t_1, t_2) \mid \begin{array}{l} t_1 \in Verb(n_1), \\ t_2 \in Verb(n_2) \end{array} \right\}$$

$$NounSim(n_1, n_2)$$

$$= \max \left\{ \text{SemSim}_t(t_1, t_2) \mid \begin{array}{l} t_1 \in \text{Noun}(n_1), \\ t_2 \in \text{Noun}(n_2) \end{array} \right\}$$

where $\text{Verb}(n)$ returns the set of verb token contained in the function names, and $\text{Noun}(n)$ returns the set of noun token contained in the function names.

The semantic similarity between a verb and a noun cannot be calculated since path similarity using WordNet can only be calculated between pairs that have the same part of speech [24]. We assume that verbs are especially important for understanding the behavior of functions, and thus we use the similarity between verb synsets when verb-based comparison is available. The semantic similarity between function names n_1 and n_2 is defined as follows.

$$\text{SemSim}(n_1, n_2) = \begin{cases} \text{VerbSim}(n_1, n_2) & \text{if } |\text{Verb}(n_1)| \geq 1 \\ & \wedge |\text{Verb}(n_2)| \geq 1 \\ \text{NounSim}(n_1, n_2) & \text{if } (|\text{Verb}(n_1)| = 0 \\ & \vee |\text{Verb}(n_2)| = 0) \\ & \wedge |\text{Noun}(n_1)| \geq 1 \\ & \wedge |\text{Noun}(n_2)| \geq 1 \\ 0 & \text{otherwise} \end{cases}$$

Adopt the semantic similarity between verbs if both n_1 and n_2 contain verb tokens, else adopt the semantic similarity between nouns if both n_1 and n_2 contain noun tokens. If there are no verb tokens and noun tokens in n_1 and n_2 , the two function names are determined to not be semantically similar.

3.2.3 The proposed similarity metric for function names

The proposed metric is a hybrid of both morphological similarity described in Sect. 3.2.1 and semantic similarity described in Sect. 3.2.2. The reason is that morphological similarity does not account for semantically similar words, and likewise, semantic similarity does not handle abbreviations or derives. The proposed metric that measures the similarity between two function names, n_1 and n_2 , is defined as follows:

$$\text{Similarity}(n_1, n_2) = \max\{\text{MorphoSim}(n_1, n_2), \text{SemSim}(n_1, n_2)\}$$

The resilience from this metric indicates the degree to which the LLM fails to capture the core meaning of the function. The more LLM predicts incorrect keywords and identifiers, the higher the resilience score is.

3.3 Requirements

The framework has two essential requirements for obfuscating transformations: program stability and traceability.

3.3.1 Program stability

In order to provide a reliable evaluation, the obfuscated program must be functionally equivalent to the original program, assuming it will be distributed to the public. In this study, we require that the non-interactive program that does not require standard input, and that the standard output be consistent before and after obfuscation. For example, a binary whose behavior is affected by random numbers, build time, or execution time does not satisfy this requirement.

3.3.2 Traceability of function names

To calculate resilience, the LLM-recovered function name must be compared with its original function name. In other words, it is necessary to keep traceability links between the obfuscated and non-obfuscated code. Therefore, code obfuscation tool must provide a function mapping between the original and obfuscated code. For example, the C obfuscator Tigress [13] version 4 embeds the information about the applied obfuscation in the obfuscated code using structured comments in JSON format, making it possible to trace the code. It should be noted that we found cases where decompiled function names may be accidentally inconsistent with function names in the obfuscated binary due to the handling of symbols such as “\$” and “_”, probably because of their internal behavior. In such cases, we consider the pair of functions with the most similar names to correspond to each other.

4. Experiment

4.1 Evaluation process

We conduct an experiment that evaluates the resilience of existing obfuscation methods against LLM-based function name recovery attacks. This experiment aims to examine the overall resilience of each obfuscation method and analyze high-resilience obfuscated code, and to discuss the impact of obfuscation methods on LLM-based function name recovery attacks.

In this experiment, we evaluate the resilience of five well-known obfuscation methods that are implemented in Tigress version 4.0.10: Encode Arithmetic (EncA), Encode Literals (EncL), Flatten (Flat), Add Opaque predicates (AddO4), and Virtualize (Virt). When AddO4 is applied, four opaque predicates are inserted into the target function. Tigress only accepts single C/C++ source files as input, and it compiles and obfuscates at the same time. Thus, when applying obfuscation, we add library include statements such as `stdlib.h` and `string.h`, or empty implementations of the main function to the code as necessary.

The dataset used in this experiment is the TheAlgorithms/C repository [27]. TheAlgorithms/C provides an open-source resource for learning data structures and algorithms implemented in the C language. Many programs

in this resource include self-checking tests and are designed not to use external libraries. The programs in this dataset implement basic algorithms and are not representative of production-scale software that would be adopted in the real world. By using simple algorithms, we exclude factors unrelated to obfuscation that could make function name recovery difficult. Since we employ Tigress as an obfuscation tool that meets the requirements for the traceability of function names described in Sect. 3.3.2, we target only programs written in C or C++. Assuming a typical binary distribution, the obfuscated programs are compiled with CMake version 3.28.3 and GCC version 12.2.0. In the compilation, we use the optimization option `-O2` and debug option `-g0`, which produces no debug information [28]. To exclude symbol table information, the `strip` command is applied to each built binary.

In the **LLM-based function name recovery** attack process, we employ the general-purpose model **Gemma3-27B-IT** [29] as the LLM. The reason for employing a general-purpose model is to **observe the extent to which MATE attackers can recover function names without specialized preparation**, such as fine-tuning for MATE attacks. Furthermore, to ensure that the results are not biased by model-specific behavior, we also conduct the same procedure on OpenAI's `gpt-oss-20b` [30] and Meta's Llama 4 (`llama-4-scout-17b-16e-instruct`) [31]. For all models, the context length is set to 31,000 and the seed value is set to 100 to ensure reproducibility. Evaluating across these three distinct LLMs gives a comprehensive evaluation of the resilience of existing obfuscation methods. The experimental results of these three models are shown in Section 4.3.1. In this paper, we mainly show the results obtained using **Gemma3-27B-IT**. Figure 2 shows the prompt template used for the function name recovery task, based on the prompt used in Shang et al.'s work [5]. The prompt includes the pseudocode of a binary as a code block. Specific prompts are generated by replacing the code block in the template with the pseudocode text of each binary. The pseudocode text for each binary is obtained by decompiling it with IDA Pro [32] (version 9.1). The tasks using the generated prompts are executed via an API server provided by LM Studio [33] 0.3.31, with the temperature parameter set to 0.0.

To calculate the resilience between the original and LLM-recovered function names, a Python (version 3.13.0) script implementing the metric introduced in Section 3.2 is used. Semantic similarity based on WordNet is calculated using the `wn` library [23] (version 0.13), which adopts the Open English WordNet 2024 Edition developed by John et al. [34] as its corpus.

4.2 Preparation

Before showing the results for the resilience of obfuscation methods, we specifically describe the selection of the programs in the dataset and the flow of the process by which the LLM recovers the function names. We also describe the process of converting abbreviated tokens obtained from

```
Please imagine you are an experienced binary reverse engineer.
The following is a decompiled source code, your task is
to understand the logic for each function and suggest a
more appropriate function name.
Your suggested function name should start with a verb.
If you determine that the function name defined is
appropriate, output the function name as it is.
You are to respond in csv format, with column 1 being
the original function name, column 2 being the function
name you guessed, and column 3 being a description of
the logic you understood.
'''C/C++
{
...

```

Fig. 2: **Prompt template** (the code block is replaced with a pseudocode text decompiled from binary)

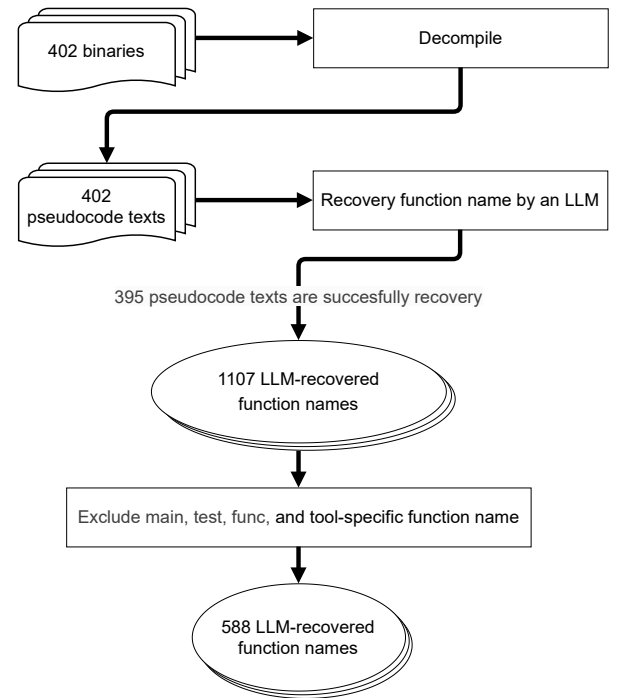


Fig. 3: **Detailed flow of the LLM-based function name recovery attack process** using the experimental dataset

function names as applied in this experiment.

The dataset consists of **189 individual programs** from TheAlgorithms/C repository. To avoid failure in applying obfuscation, we exclude the programs in the `developer.tools` directory that use custom header files. To satisfy the requirement of program stability described in Sect. 3.3.1, we pick up **67 programs** which do not require standard input (i.e., non-interactive) and have consistent results of standard output between obfuscation and non-obfuscation.

In case of employing the **Gemma3-27B-IT**, Figure 3 illustrates the detailed flow of the LLM-based function name recovery attack process shown in Fig. 1, using the experi-

Table 1: Number of LLM-recovered function names and number of resilience calculation targets (logic-relevant functions) for each obfuscation method

Obfuscation method	LLM-recovered function names	Resilience calculation target
Add04	177	86
EncA	173	91
EncL	209	115
Flat	185	103
Virt	197	111
Non-Obfuscated	166	82
Total	1107	588

Table 2: Token conversion table for this experiment

Original token	Converted (unabbreviated) token
fib	fibonacci
int	integer
poly	polynomial
prod	product
quat	quaternion
vec	vector

mental dataset. First, we obtained 402 binaries by applying six transformations to each of the 67 programs in the dataset. The transformations consist of five obfuscation methods and a non-obfuscation case. Then, 402 pseudocode texts were obtained by decompiling 402 binaries. The LLM recovered function name sets from 395 pseudocode texts, while it failed to recover function name sets from the remaining 7 pseudocode texts, which means that the function name recovery task was not completed for these 7 cases. There are a total of 1107 function names in the 395 pseudocode texts. From these, we exclude function names that do not match the logic at the original level: containing `main`, `test`, `matching_func`, or starting with `_TIG`. The functions whose names start with `_TIG` were inserted by Tigress. Table 1 shows the number of LLM-recovered function names for each obfuscation method and the number of resilience calculation targets (logic-relevant functions) for each obfuscation method. Finally, we calculated resilience for 588 function names.

In addition, in this experiment, tokens that indicate abbreviations were replaced with their corresponding unabbreviated tokens to improve the accuracy of morphological similarity and to avoid deriving incorrect synonyms. The converted tokens are shown in Table 2.

4.3 Results and discussions

4.3.1 Overall resilience of each obfuscation method

To examine the overall resilience of each obfuscation

Table 3: Number of function names for each similarity measure in the similarity calculation for each obfuscation method (Non-similar: Resilience = 1.00)

Obfuscation method	<i>MorphoSim</i>	<i>SemSim</i>	Non-similar	Total
Add04	39	40	7	86
EncA	66	23	2	91
EncL	47	63	5	115
Flat	66	34	3	103
Virt	23	82	6	111
Non-Obfuscated	55	19	8	82

method, we calculate the resilience of each function using the proposed metric and observe the tendency of resilience for each obfuscation method.

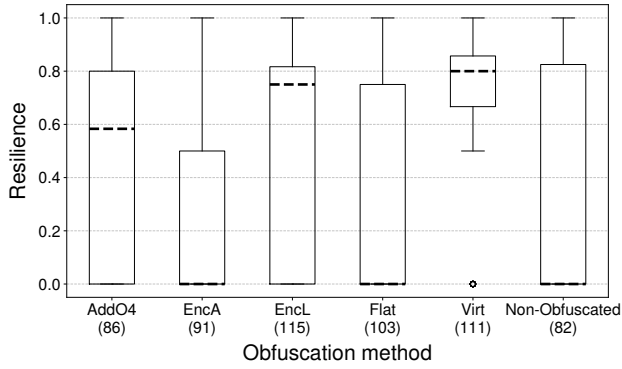
Table 3 shows the number of function names for each similarity measure in the similarity calculation described in Sect. 3.2, for each obfuscation method, obtained using Gemma3-27B-IT. The number of function names calculated by *MorphoSim*, that is, have morphologically similar tokens is the lowest in *Virt* with 23 and the highest in *EncA* and *Flat* with 66.

Figure 4 shows box plots of the resilience of each obfuscation method against the LLM-based function name recovery attacks. Figure 4(a), Figure 4(b), and Figure 4(c) show the results when Gemma3-27B-IT, gpt-oss-20b, and llama-4-scout-17b-16e-instruct are employed as the LLM, respectively. The number in parentheses in each figure indicates the number of target functions. The resilience of *Virt* has the highest median value among all models employed in this experiment. These results imply that *Virt* is promising for protecting against LLM-based function name recovery attacks. On the other hand, the resilience of *Add04*, *EncA*, *EncL*, and *Flat* varies depending on the model. In particular, the resilience of *EncA* shows a low median among all models. The resilience of *Add04*, *EncL*, and *Flat* show a low median in Figure 4(b), while showing a high median in either Figure 4(a) or Figure 4(c). These methods are considered to have only limited resilience against LLM-based function name recovery attacks. In other words, programs obfuscated by *Add04*, which inserts opaque predicates to the program, *EncA*, which replaces integer arithmetic in the program with more complex expressions [13], *EncL*, which replaces literals with less obvious expressions [13], or *Flat*, which flattens the control flow of the program, tend to be relatively easy for the LLM to understand.

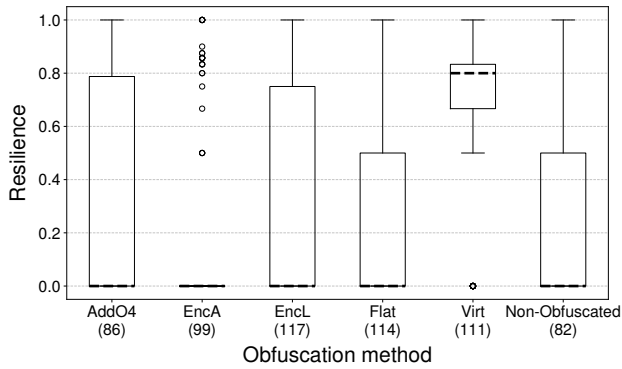
4.3.2 Analysis of high-resilience obfuscated code

We then focus on obfuscated code whose resilience is higher than that of the corresponding non-obfuscated code, and discuss the impact of obfuscation methods on LLM-based function name recovery attacks.

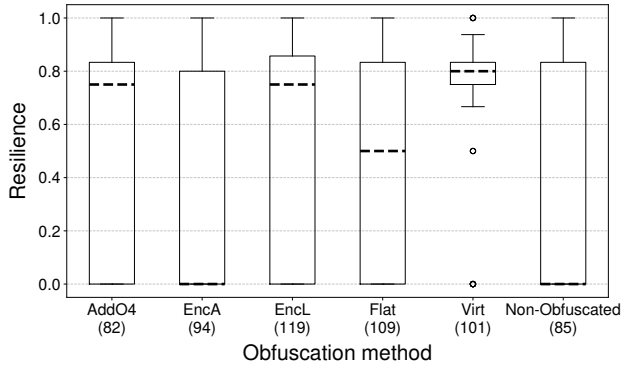
We consider a function to be a *high-resilience function* if the resilience of its code is higher than that of the non-



(a) Gemma3-27B-IT



(b) gpt-oss-20b



(c) llama-4-scout-17b-16e-instruct

Fig. 4: Overall resilience of each obfuscation method against the LLM-based function name recovery attacks

obfuscated version. To determine whether resilience is high, it is necessary to compare it with that of Non-Obfuscated. Therefore, for each obfuscation method, only the function names that are common to Non-Obfuscated are extracted. As a result, 296 of the 588 targets for resilience calculation in the case using Gemma3-27B-IT were analyzed. The result of the analysis is shown in Table 4. For Virt, 52.5% of functions were evaluated as high-resilience functions, which implies that it is relatively promising for preventing the func-

Table 4: Ratio of high-resilience function for each obfuscation method

Obfuscation method	Ratio of high-resilience functions (%)	Total
AddO4	12.2	41
EncA	7.5	67
EncL	32.8	61
Flat	13.2	68
Virt	52.5	59

tion name recovery attack.

Table 5 shows the list of the LLM-recovered function names of the functions that were evaluated as high-resilience functions for each obfuscation method, their original function names, and the names of the source files in which the functions are implemented. The recovered function names of Virt, which is the most resistant obfuscation method, include the words related to bytecode (e.g. `execute_bytecode`), indicating that the original function names have been concealed. On the other hand, Table 5 includes some results showing that the LLM seems to have understood the code and recovered the function names correctly. For example, the function `evaluate` in the program whose source file name is `postfix_evaluation.c` was recovered as `process_postfix_token`. In this case, the recovered name was not evaluated as highly similar to the original since source file names are not considered in this experiment. In the future, we believe that considering multiple perspectives, including source file names in addition to function names and the context in which functions are used in the programs, will enable a more detailed evaluation.

RQ: How resilient are software-based obfuscation methods against LLM-based function name recovery in MATE attacks? Based on the experimental results evaluating the resilience of five well-known obfuscation methods, we observe that the obfuscation method based on code virtualization has the highest resilience, and it is promising for protecting programs against LLM-based function name recovery attacks. In contrast, the obfuscation method based on encoding arithmetic has the lowest resilience. Furthermore, the resilience of several obfuscation methods including opaque predicates insertion, encoding literals, and control flow flattening varies depending on the model.

5. Discussion

5.1 Effectiveness of the Proposed Method

The proposed method determines that two given function names are similar when morphologically similar tokens exist, or when semantically similar verb or noun tokens exist. This criterion is considerably more lenient than general similarity measures such as Longest Common Subsequence (LCS), the Jaccard coefficient, and, the Simpson coefficient. The LCS is a string-based metric, while the Jac-

Table 5: LLM-recovered function names evaluated as high-resilience functions for each obfuscation method, their original function names, and the names of source files in which the functions are implemented

Source file name	Function name	Obfuscation method	LLM recovered function name
affine.c	affine_decrypt	EnCL	process_and_validate_data
		Virt	execute_instructions
	inverse_key	Add04	run_secondary_tests
		EnCL	calculate_value
		Virt	process_data_stream
alaw.c	encode	EnCL	initialize_data
armstrong_number.c	power	Flat	calculate_recursive
binary_to_decimal.c	convert_to_decimal	EnCL	calculate_value
		Virt	interpret_bytecode
bisection_method.c	bisection	EnCA	find_root
		Virt	solve_equation
		Add04	allocate_and_initialize_nodes
	sign	EnCL	allocate_and_manipulate_memory
		Flat	calculate_value
		Virt	evaluate_expression
multibubble_sort_2.c	bubble_sort	Flat	transform_array
		Virt	decode_and_execute
bucket_sort.c	BucketSort	Virt	process_and_print_array
	InsertionSort	EnCL	create_node
		Virt	process_complex_data
	print	Flat	display_array
euclidean_algorithm_extended.c	extended_euclidean_algorithm	EnCL	calculate_quotient_remainder
		Virt	validate_and_transform_data
fibonacci_search.c	fibMonaccianSearch	EnCA	find_index
		Virt	interpret_and_process_data
heap_sort_2.c	heapifyDown	EnCL	compare_and_swap
		Flat	swap_and_check
		Virt	process_data
insertion_sort.c	insertionSort	EnCL	manage_free_list
		Virt	process_array
lu_decompose.c	lu_decomposition	Flat	calculate_arrays
matrix_chain_order.c	printSolution	Add04	compute_matrix_chain_order
non_preemptive_priority_scheduling.c	show_list	EnCA	print_scheduling_results
odd_even_sort.c	oddEvenSort	Flat	perform_bit_rotations_and_comparisons
		Virt	process_data_stream
other_binary_search.c	binarySearch	Virt	analyze_data
partition_sort.c	partition	EnCL	sort_linked_list
		Virt	process_data_complex
	patienceSort	EnCL	process_data_and_compare
		Virt	process_data

Table 5 — continued from previous page

Source file name	Function name	Obfuscation method	LLM recovered function name
poly_add.c	create_polynomial	EncL	allocate_and_initialize_node
		Virt	process_data_stream
	display_polynomial	EncL	print_linked_list_data
		Virt	format_and_print_data
	poly_add	EncL	create_linked_list
		Virt	manipulate_data_structures
postfix_evaluation.c	evaluate	Flat	process_postfix_token
prime.c	isPrime	Virt	process_instruction
problem_10-sol1.c	is_prime	AddO4	calculate_root
		EncL	calculate_sqrt_condition
		Virt	interpret_bytecode
	sum_of_primes	EncA	calculate_complex_value
		EncL	compute_recursive_value
		Virt	execute_bytecode
quaternions.c	euler_from_quat	Virt	decode_data
	quat_from_euler	Virt	process_data
	quaternion_multiply	EncL	allocate_and_link_nodes
		Virt	decode_data
rot13.c	rot13	EncL	apply_rotational_cipher
selection_sort.c	selectionSort	Virt	decode_and_execute_instructions
ternary_search.c	ternarySearch	Flat	find_index
		Virt	interpret_bytecode
union_find.c	find	EncA	calculate_and_update_value
		EncL	manipulate_array_element
		Virt	interpret_bytecode
vectors_3d.c	get_angle	Virt	calculate_dot_product
	print_vector	EncL	format_string
		Virt	process_data
	unit_vec	AddO4	calculate_distance
		Virt	process_memory

card coefficient and the Simpson coefficient are set-based ($T(n)$ -based) metrics. Table 6 shows 10 examples of similarity scores obtained using the proposed method, LCS-based similarity, the Jaccard coefficient, and, the Simpson coefficient. For example, when the LLM-recovered function name is “`sort_and_compare_vectors`” for a function named “`bubbleSort`”, the proposed method obtained a high similarity score (1.00) because morphologically similar tokens exist. The LCS-based similarity was 0.17 because common substrings are short. The Jaccard coefficient and the Simpson coefficient were 0.20 and 0.50, respectively, since the names contain different tokens. The proposed method tend to result in higher similarity scores than those textual similarity measures; therefore, it is more sensitive to potential risks arising from the possibility that LLMs may reveal semantic clues about the program.

5.2 Practical Applicability

This framework enables developers of obfuscation methods to develop new versions of existing techniques and evaluate their resilience prior to release. For example, developers can explore improvements to the `EncL` and `Flat` algorithms, which exhibited low resilience against some LLMs, or further advance `Virt`, which demonstrated consistently high resilience across all models in our experiments.

Users of obfuscation methods can also benefit from this framework. It enables them to automatically compare the resilience of multiple obfuscation methods and select the most effective one for their programs. In addition, the framework allows users to assess, on a per-function basis, how easily each function can be understood by LLMs. Based on this assessment, users can take targeted countermeasures to reduce LLM comprehension, such as applying multiple obfuscation techniques to selected functions.

6. Threats to Validity

6.1 Internal Validity

One threat to the internal validity of our experiment is that we employ only one type of general-purpose LLM model. As described in Sect. 4.1, we use a general-purpose model to observe the extent to which MATE attackers can recover function names without specialized preparation such as fine-tuning for MATE attacks. While evaluating multiple models may lead to more practical results, our study is a first step towards understanding the resilience risks posed by general-purpose models.

Another internal threat is that the proposed similarity metric may determine that a word and its antonym are morphologically similar and return a similarity score of 1. Specifically, this threat can occur when the first or last character of the words matches and the Levenshtein similarity is greater than 0.6. For example, words such as “`encrypt`” and “`decrypt`,” or “`encode`” and “`decode`,” have opposite meanings, but the proposed similarity metric returns 1,

treating them as morphologically similar. We observed four cases of such false detections in the experimental data. The proposed metric could be further improved to return more appropriate values in cases where words are morphologically similar but not semantically similar. At the same time, the metric correctly handles cases where original forms and their variants (e.g., “`send`” and “`sent`”) or words with prefixes (e.g., “`str`” and “`cstr`,” “`gets`” and “`fgets`”) are compared. In this paper, we prioritize maximizing the identification of morphologically similar words rather than minimizing false positives.

A further internal threat is that the calculation of resilience does not take into consider the naming based on the context of the entire program. In software development, function names are often based on the role and structure of the function within the program as a whole, rather than on the implementation details of the function. Therefore, the proposed framework may overestimate resilience even when an LLM recovers the appropriate function names. We believe that this impact can be mitigated by including the program context in the calculation of resilience. However, if the wrong context is considered, it could have a significant impact on the calculation of resilience. Therefore, the calculation of resilience in this experiment is restricted to converting abbreviated tokens to appropriate names, without giving much consideration to context.

6.2 External Validity

One threat to external validity is the limited scale of the dataset. Although it satisfies the requirements for program stability and function name traceability described in Sect. 3.3, resilience evaluation results may differ with larger datasets. This paper analyzes the resilience of functions for each obfuscation method using a small but diverse dataset, selected to minimize the influence of factors unrelated to obfuscation.

7. Conclusion and Future work

This paper has proposed a framework for evaluating the resilience of obfuscated code against LLM-based function name recovery MATE attacks. Using this framework, we conducted an experiment to evaluate the resilience of five well-known obfuscation methods against such attacks. The experimental results revealed that code virtualization has the highest resilience, while encode arithmetic has the lowest resilience and the resilience of several obfuscation methods including opaque predicates insertion, encode literals, and control flow flattening varies depending on the model.

In future work, an important issue is to expand the evaluation framework to assess the resilience of obfuscation methods in more practical contexts. Examples include employing fine-tuned models for MATE attacks and calculating resilience in a way that considers the naming of function names based on the role in the whole program rather than their implementations. In particular, calculating resilience

Table 6: 10 examples of similarity scores obtained using the proposed method, LCS-based similarity, the Jaccard coefficient, and the Simpson coefficient

Function name	Obfuscation method	LLM recovered function name	Similarity	LCS-based similarity	Jaccard coefficient	Simpson coefficient
selectionSort	Add04	sort_and_process_array	1.00	0.27	0.20	0.50
binarysearch2	Add04	search_sorted_array	1.00	0.37	0.20	0.33
bubbleSort	EncA	sort_and_compare_vectors	1.00	0.17	0.20	0.50
count	EncA	calculate_sum	1.00	0.23	0.00	0.00
join	EncL	allocate_and_link_node	1.00	0.14	0.00	0.00
heapSort	EncL	sort_and_compare	1.00	0.25	0.25	0.50
fibMonaccianSearch	Flat	search_for_index	1.00	0.27	0.20	0.33
affine.decrypt	Flat	encrypt_string	1.00	0.43	0.00	0.00
partitionSort	Virt	sort_array	1.00	0.29	0.33	0.50
number_of_paths	Virt	compute_diagonal_paths	1.00	0.45	0.20	0.33

using embedding models such as codeBERT[†] is an interesting challenge. In addition, we believe that building a continuous evaluation system enables state-of-the-art evaluation that keeps pace with the growth of LLMs. One of the key challenges is to prepare a dataset that is suitable for such evaluation. The development of new obfuscation methods, or combinations of existing obfuscation methods, that are resilient to LLM-based MATE attacks is also an interesting issue.

Acknowledgments

This research has been supported by JSPS KAKENHI Nos. JP22K11986 and JP23K16862.

References

- [1] Hex-rays, “IDA Decompilers.” <https://hex-rays.com/decompiler/>.
- [2] National Security Agency, “Ghidra.” <https://www.nsa.gov/ghidra>.
- [3] Vector 35, “Binary Ninja.” <https://binary.ninja/>.
- [4] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, “Using an LLM to help with code understanding,” Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Association for Computing Machinery, 2024.
- [5] X. Shang, S. Cheng, G. Chen, Y. Zhang, L. Hu, X. Yu, G. Li, W. Zhang, and N. Yu, “How far have we gone in binary code understanding using large language models,” 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp.1–12, IEEE Computer Society, Oct. 2024.
- [6] L. Jiang, X. Jin, and Z. Lin, “Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs,” 32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025, The Internet Society, 2025.
- [7] B. De Sutter, S. Schrittwieser, B. Coppens, and P. Kochberger, “Evaluation methodologies in software protection research,” ACM Computing Surveys, vol.57, no.4, Dec. 2024.
- [8] C. Collberg and J. Nagra, Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Program Protection, Addison-Wesley Professional, 2009.
- [9] S. Banescu and A. Pretschner, “A tutorial on software obfuscation,” Advances in Computers, vol.108, pp.283–353, Elsevier, 2018.
- [10] M. Ceccato, M. Di Penta, P. Falcarin, F. Ricca, M. Torchiano, and P. Tonella, “A family of experiments to assess the effectiveness and efficiency of source code obfuscation techniques,” Empirical Software Engineering, vol.19, no.4, pp.1040–1074, Aug 2014.
- [11] S. Banescu, C. Collberg, V. Ganesh, Z. Newsham, and A. Pretschner, “Code obfuscation against symbolic execution attacks,” Proceedings of the 32nd Annual Conference on Computer Security Applications, ACSAC ’16, pp.189–200, Dec. 2016.
- [12] S. Banescu, C. Collberg, and A. Pretschner, “Predicting the resilience of obfuscated code against symbolic execution attacks via machine learning,” Proceedings of the 26th USENIX Conference on Security Symposium, SEC’17, pp.661–678, USENIX Association, 2017.
- [13] C. Collberg, “The Tigress C obfuscator.” <https://tigress.wtf>.
- [14] Y. Kanzaki, A. Monden, and C. Collberg, “Code Artificiality: A Metric for the Code Stealth Based on an N-Gram Model,” Proceedings of 2015 IEEE/ACM 1st International Workshop on Software Protection (SPRO ’15), pp.31–37, IEEE Computer Society, 2015.
- [15] S.A. Ebad, A.A. Darem, and J.H. Abawajy, “Measuring Software Obfuscation Quality—A Systematic Literature Review,” IEEE Access, vol.9, pp.99024–99038, 2021.
- [16] Y. Isobe and H. Tamada, “Are identifier renaming methods secure? – an evaluation focuses on opcodes using random forest–,” Proceedings of 19th ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD 2018), July 2018.
- [17] C. Patsakis, F. Casino, and N. Lykousas, “Assessing LLMs in malicious code deobfuscation of real-world malware campaigns,” Expert Systems with Applications, vol.256, Dec. 2024.
- [18] D. Beste, G. Menguy, H. Hajipour, M. Fritz, A.E. Cinà, S. Bardin, T. Holz, T. Eisenhofer, and L. Schönherr, “Exploring the potential of LLMs for code deobfuscation,” Detection of Intrusions and Malware, and Vulnerability Assessment: 22nd International Conference, DIMVA 2025, Graz, Austria, July 9–11, 2025, Proceedings, Part I, Berlin, Heidelberg, pp.267–286, Springer-Verlag, 2025.
- [19] C. Fang, N. Miao, S. Srivastav, J. Liu, R. Zhang, R. Fang, Asmita, R. Tsang, N. Nazari, H. Wang, and H. Homayoun, “Large language models for code analysis: Do LLMs really do their job?,” 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, pp.829–846, USENIX Association, Aug. 2024.
- [20] Y. Kashiwabara, T. Ishio, H. Hata, and K. Inoue, “Method Verb

[†]microsoft/CodeBERT: <https://github.com/microsoft/CodeBERT>

- Recommendation Using Association Rule Mining in a Set of Existing Projects,” IEICE Transactions on Information, vol.E98-D, no.3, pp.627–636, Mar. 2015.
- [21] H. Gao, S. Cheng, Y. Xue, and W. Zhang, “A lightweight framework for function name reassignment based on large-scale stripped binaries,” Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp.607–619, Association for Computing Machinery, 2021.
- [22] “ztane/python-Levenshtein.” <https://github.com/ztane/python-Levenshtein>.
- [23] “wn.similarity - wn 0.13.0 documentation.” <https://wn.readthedocs.io/en/latest/api/wn.similarity.html>.
- [24] T. Pedersen, S. Patwardhan, and J. Michelizzi, “WordNet: similarity - measuring the relatedness of concepts,” Proceedings of the 19th National Conference on Artificial Intelligence, pp.1024–1025, AAAI Press, 2004.
- [25] “NLTK :: Sample usage for wordnet.” <https://www.nltk.org/howto/wordnet.html>.
- [26] P. University, “WordNet.” <https://wordnet.princeton.edu>.
- [27] “TheAlgorithms/C.” <https://github.com/TheAlgorithms/C>.
- [28] “Debugging Options (Using the GNU Compiler Collection (GCC)).” <https://gcc.gnu.org/onlinedocs/gcc-12.2.0/gcc/Debugging-Options.html>.
- [29] Gemma Team, “Gemma 3 technical report.” <https://arxiv.org/abs/2503.19786>, March 2025.
- [30] OpenAI, “gpt-oss-120b & gpt-oss-20b Model Card.” <https://arxiv.org/abs/2508.10925>, Oct. 2025.
- [31] “Llama 4 — Model Cards and Prompt formats.” <https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/>.
- [32] Hex-rays, “IDA Pro.” <https://hex-rays.com/ida-pro>.
- [33] “LM Studio.” <https://lmstudio.ai>.
- [34] J.P. McCrae, A. Rademaker, F. Bond, E. Rudnicka, and C. Fellbaum, “English WordNet 2019 – an open-source WordNet for English,” Proceedings of the 10th Global Wordnet Conference, ed. P. Vossen and C. Fellbaum, Wrocław, Poland, pp.245–252, Global Wordnet Association, July 2019.



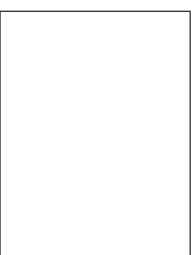
Takashi Ishio received the Ph.D. degree in information science and technology from Osaka University in 2006. He was a JSPS Research Fellow from 2006-2007. He was an assistant professor at Osaka University from 2007-2017. He was an associate professor at Nara Institute of Science and Technology from 2017-2023. He is now a professor at Future University Hakodate from 2023. His research interests include program analysis, program comprehension, and software reuse.



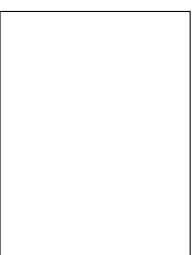
Kazumasa Shimari received the Ph.D. degree in information science and technology from Osaka University in 2022. He is now an assistant professor at Nara Institute of Science and Technology. His research interests include program analysis and program comprehension.



Kenichi Matsumoto is a professor in the Division of Information Science at Nara Institute of Science and Technology, Japan. His research interests include software measurement and software process. Matsumoto received a Ph.D. in Engineering from Osaka University. He is a senior member of IEEE, and a fellow of the IEICE, and the IPSJ. Contact him at matumoto@is.naist.jp.



Tetsuya Kitaoka received the M.E. degree from Nara Institute of Science and Technology in 2023. He is now a doctoral student in the Graduate School of Science and Technology at Nara Institute of Science and Technology from 2023. His research interests include code obfuscation technology evaluation.



Yuichiro Kanzaki received the B.E. degree from Kobe University in 2001, and the M.E. and D.E. degrees from Nara Institute of Science and Technology, in 2003 and 2006, respectively. He is currently an associate professor at National Institute of Technology, Kumamoto College. His research interests include software protection and program analysis.